INDEX

Sr.No	Problem Statement	Page No	Sign
UNIT 1: Introduction to Unsupervised Learning			
1	Write a python program to Prepare a sample dataset of customers containing parameters such as Age, Annual Income, and Spending Score. Apply the K-Means Clustering algorithm to group customers into different segments		
2	Write a python program for Prepare a sample dataset containing coordinates of different points. Calculate the Euclidean Distance and Manhattan Distance between the points and compare the results.		
3	Write a python program to create a dataset with multiple features such as student marks in different subjects. Apply Principal Component Analysis (PCA) to reduce the dimensions of the dataset.		
4	Write a python program Prepare a dataset containing features of different flowers such as petal length and petal width. Apply the K-Means Clustering algorithm to group similar flowers into clusters.		
UNIT 2: Clustering Algorithms			
Sr.No	**Problem Statement**	**Page No**	**Sign**
5	Write a python program to Create a sample dataset and apply Hierarchical Clustering. Draw and interpret the Dendrogram. (Bottom to up)		
6	Write a python program to Create a sample dataset and visualize the clusters using Matplotlib. Show different clusters with different markers.		
7	Write a python program to Train a K-Means clustering model using Scikit-learn and display the cluster labels for the given dataset.		
8	Write a python program to Create a sample dataset of customers with Age and Income. Apply K-Means Clustering and group the customers into different clusters.		
Unit 3: Dimensionality Reduction Techniques			
Sr.No	**Problem Statement**	**Page No**	**Sign**
9	Write a python program make a dataset with multiple features and apply PCA to reduce the number of features from 4 to 2 using Scikit-learn.		
10	Write a python program to Apply PCA to a dataset and use a scatter plot to visualize the data before and after dimensionality reduction.		
11	Write a python program Create a dataset containing two-dimensional points and display the points using a Matplotlib scatter plot.		

Sr.No	Problem Statement	Page No	Sign
12	Write a python program to Calculate and display the variance of different features in a small dataset using Python.		
UNIT 4: Feature Engineering & Encoding			
Sr.No	**Problem Statement**	**Page No**	**Sign**
13	Write a python program create a simple dataset such as house size and price to build a neural network model that predicts the price of a house.		
14	Write a python progarn to Create a simple artificial neuron in Python using input values, weights, and bias. Calculate the output of the neuron.		
15	Write a Python program to apply the Sigmoid, ReLU, and Tanh activation functions on given input values and display their outputs.		
16	Write a python program to Create a simple feedforward neural network using Keras with an input layer, one hidden layer, and an output layer.		
17	Write a python program in program Use a small dataset of student marks and attendance to build a simple neural network that predicts whether a student will pass or fail.		
18	Write a python program to Create a simple neural network using Keras to classify data into two classes and display the predicted output.		
19	Write a payprogram program for Train a simple neural network model using a sample dataset and display the loss value after each training epoch.		

1. Write a python program to Prepare a sample dataset of customers containing parameters such as Age, Annual Income, and Spending Score. Apply the K-Means Clustering algorithm to group customers into different segments.

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans


data = {

    'Age': [20, 22, 25, 28, 35, 40, 45, 50, 55, 60],

    'Annual_Income': [20, 25, 30, 35, 50, 55, 60, 70, 75, 80],

    'Spending_Score': [80, 75, 70, 65, 60, 50, 45, 40, 30, 25]

}

df = pd.DataFrame(data)


X = df[['Age', 'Annual_Income', 'Spending_Score']]

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

df['Cluster'] = kmeans.fit_predict(X)

print(df)


print("\nCluster Centers:")

print(kmeans.cluster_centers_)
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q1.py"
   Age  Annual_Income  Spending_Score  Cluster
0   20             20              80        1
1   22             25              75        1
2   25             30              70        1
3   28             35              65        1
4   35             50              60        2
5   40             55              50        2
6   45             60              45        2
7   50             70              40        0
8   55             75              30        0
9   60             80              25        0

Cluster Centers:
[[55.         75.         31.66666667]
 [23.75       27.5        72.5       ]
 [40.         55.         51.66666667]]
```

2. Write a python program for Prepare a sample dataset containing coordinates of different points. Calculate the Euclidean Distance and Manhattan Distance between the points and compare the results.

Code:

```
import math


x1 = float(input("Enter x-coordinate of the first point: "))

y1 = float(input("Enter y-coordinate of the first point: "))


x2 = float(input("Enter x-coordinate of the second point: "))

y2 = float(input("Enter y-coordinate of the second point: "))


euclidean = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)


manhattan = abs(x2 - x1) + abs(y2 - y1)


print("Point 1:", (x1, y1))

print("Point 2:", (x2, y2))


print("Euclidean Distance:", euclidean)

print("Manhattan Distance:", manhattan)
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q2.py"
Enter x-coordinate of the first point: 45
Enter y-coordinate of the first point: 79
Enter x-coordinate of the second point: 16
Enter y-coordinate of the second point: 37
Point 1: (45.0, 79.0)
Point 2: (16.0, 37.0)
Euclidean Distance: 51.03920062069938
Manhattan Distance: 71.0
```

3.Write a python program to create a dataset with multiple features such as student marks in different subjects. Apply Principal Component Analysis (PCA) to reduce the dimensions of the dataset.

Code:

```
import pandas as pd

from sklearn.decomposition import PCA


data = {

    "Maths": [80, 75, 90, 60, 85],

    "Science": [85, 70, 95, 65, 80],

    "Gujarati": [75, 80, 85, 70, 90]

}


df = pd.DataFrame(data)

print("Original Data:")

print(df)


pca = PCA(n_components=1)

New_Data = pca.fit_transform(df)

print("\nNew Data:")

print(New_Data)
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Dat
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q3.py"
Original Data:
   Maths  Science  Gujarati
0     80       85        75
1     75       70        80
2     90       95        85
3     60       65        70
4     85       80        90

New Data:
[[  3.47966225]
 [ -7.89972444]
 [ 20.26130081]
 [-24.72315274]
 [  8.88191412]]
```

4.Write a python program Prepare a dataset containing features of different flowers such as petal length and petal width. Apply the K-Means Clustering algorithm to group similar flowers into clusters.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans


data = {
    'Petal_Length': [1.4, 1.5, 1.3, 4.5, 4.7, 4.6, 5.5, 5.8, 6.0],
    'Petal_Width': [0.2, 0.3, 0.2, 1.5, 1.4, 1.5, 2.0, 2.2, 2.1]
}


df = pd.DataFrame(data)


X = df[['Petal_Length', 'Petal_Width']]

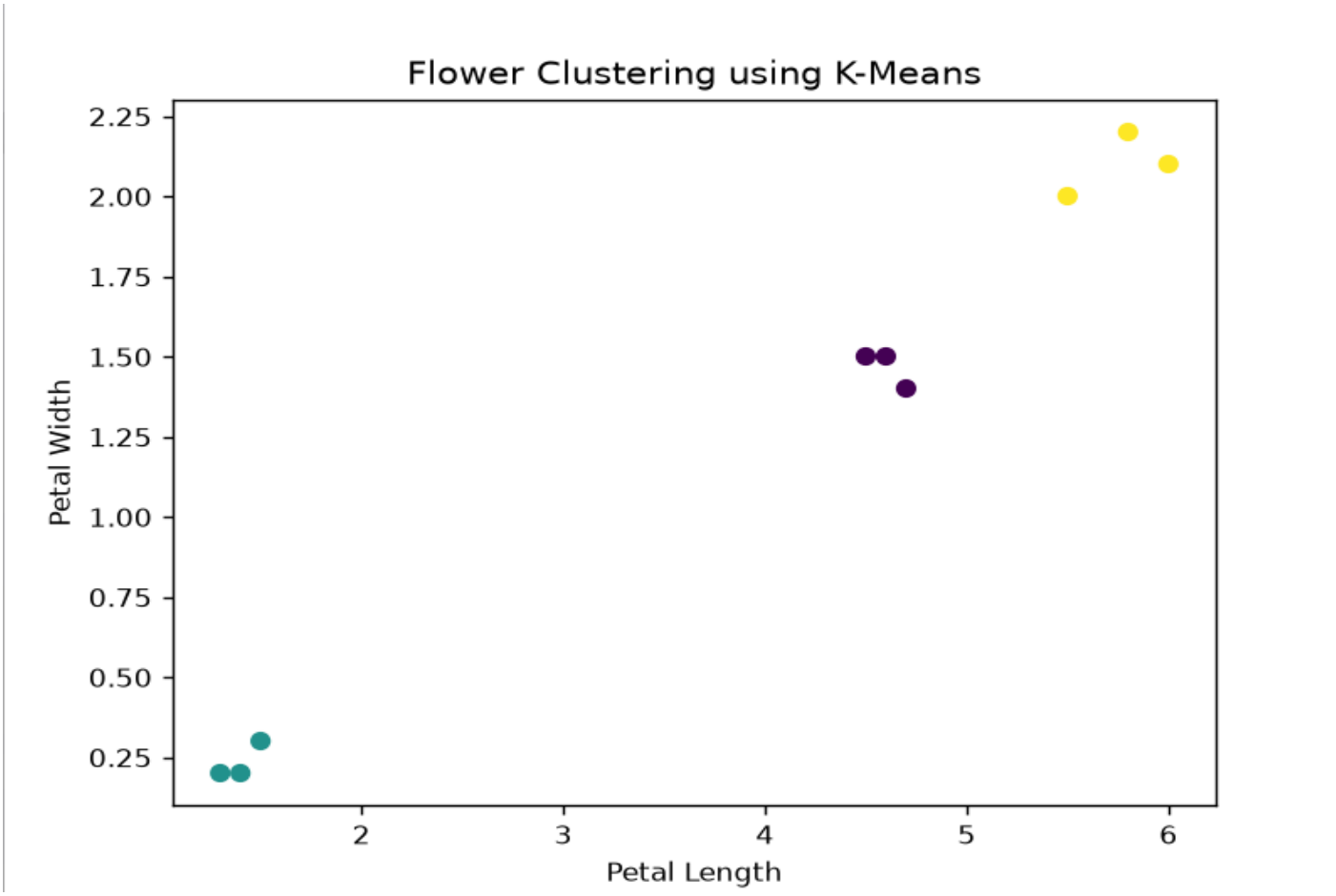

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X)
print(df)


plt.scatter(df['Petal_Length'], df['Petal_Width'],c=df['Cluster'])


plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("Flower Clustering using K-Means")
plt.show()
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q4.py"
   Petal_Length  Petal_Width  Cluster
0           1.4          0.2        1
1           1.5          0.3        1
2           1.3          0.2        1
3           4.5          1.5        0
4           4.7          1.4        0
5           4.6          1.5        0
6           5.5          2.0        2
7           5.8          2.2        2
8           6.0          2.1        2
```



5.Write a python program to Create a sample dataset and apply Hierarchical Clustering. Draw and interpret the Dendrogram. (Bottom to up).

Code:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.cluster.hierarchy import dendrogram, linkage

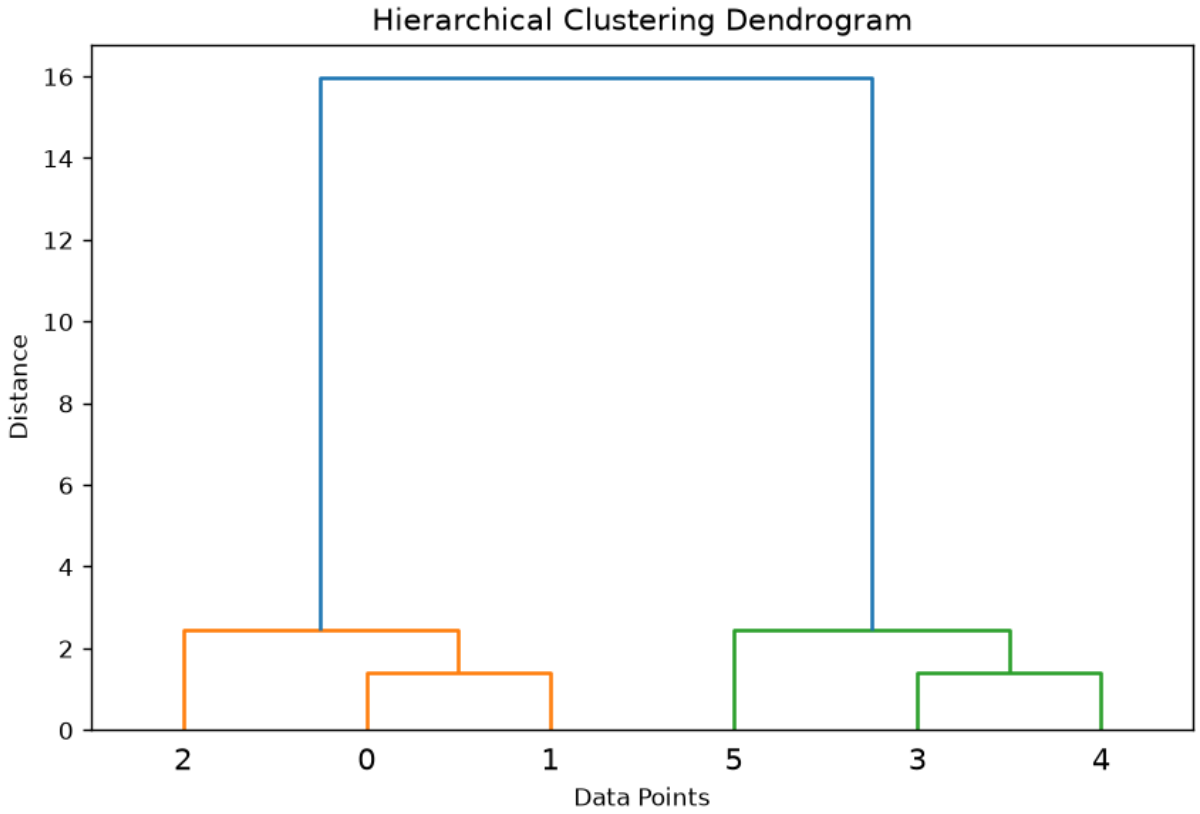

X = np.array([[1, 2], [2, 3], [3, 4], [8, 8], [9, 9], [10, 10]])


Z = linkage(X, method='ward')
```

```
plt.figure(figsize=(8, 5))
dendrogram(Z)


plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
```

Output:



6.Write a python program to Create a sample dataset and visualize the clusters using Matplotlib. Show different clusters with different markers.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

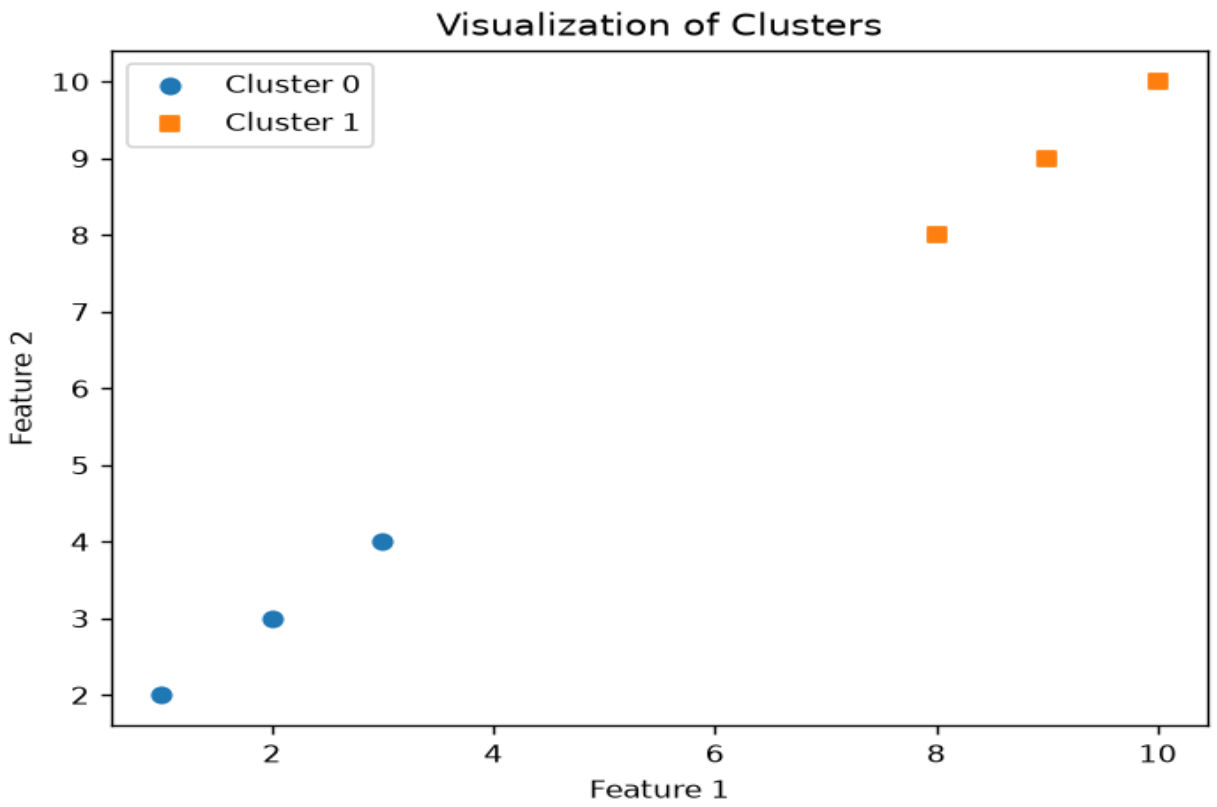

X = np.array([[1, 2], [2, 3], [3, 4], [8, 8], [9, 9], [10, 10]])
```

```
labels = [0, 0, 0, 1, 1, 1]


for cluster in [0, 1]:
    points = X[np.array(labels) == cluster]
    plt.scatter(
        points[:, 0],
        points[:, 1],
        marker='o' if cluster == 0 else 's',
        label=f'Cluster {cluster}'
    )


plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Visualization of Clusters")
plt.legend()
plt.show()
```

Output:



7.Write a python program to Train a K-Means clustering model using Scikit-learn and display the cluster labels for the given dataset.

Code:

```
import numpy as np

from sklearn.cluster import KMeans


X = np.array([
    [1, 2],
    [2, 3],
    [3, 4],
    [8, 8],
    [9, 9],
    [10, 10]
])


model = KMeans(n_clusters=2, random_state=42, n_init=10)

model.fit(X)

print("Cluster Labels:")

print(model.labels_)
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q7.py"
Cluster Labels:
[0 0 0 1 1 1]
```

8.Write a python program to Create a sample dataset of customers with Age and Income. Apply K-Means Clustering and group the customers into different clusters.

Code:

```
import pandas as pd

from sklearn.cluster import KMeans

data ={

    'Age': [25, 30, 45, 50, 22, 35, 48, 52],

    'Income': [20000, 30000, 80000, 85000, 22000, 31000, 78000,90000]
```

```
}

df = pd.DataFrame(data)

kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)

df['Cluster'] = kmeans.fit_predict(df)

print(df)
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q8.py"
   Age  Income  Cluster
0   25   20000        1
1   30   30000        1
2   45   80000        0
3   50   85000        0
4   22   22000        1
5   35   31000        1
6   48   78000        0
7   52   90000        0
```

9.Write a python program make a dataset with multiple features and apply PCA to reduce the number of features from 4 to 2 using Scikit-learn.

Code:

```
import numpy as np

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA


data = np.array([

    [2, 4, 6, 8],

    [10, 12, 14, 16],

    [18, 20, 22, 24],

    [26, 28, 30, 32],

    [34, 36, 38, 40]

])


#df = pd.DataFrame(data)

print("Original Data:")

print(data)
```

```
pca = PCA(n_components=2)

New_Data = pca.fit_transform(data)

print("\nNew Data:")

print(New_Data)
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q9.py"
Original Data:
[[ 2  4  6  8]
 [10 12 14 16]
 [18 20 22 24]
 [26 28 30 32]
 [34 36 38 40]]

New Data:
[[-3.20000000e+01 -2.91885213e-15]
 [-1.60000000e+01  9.72950711e-16]
 [ 0.00000000e+00 -0.00000000e+00]
 [ 1.60000000e+01 -9.72950711e-16]
 [ 3.20000000e+01 -1.94590142e-15]]
```

10.Write a python program to Apply PCA to a dataset and use a scatter plot to visualize the data before and after dimensionality reduction.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

from sklearn.preprocessing import StandardScaler


X = np.array([

    [1, 2],

    [2, 3],

    [3, 4],

    [8, 8],

    [9, 9],

    [10, 10]

])


scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

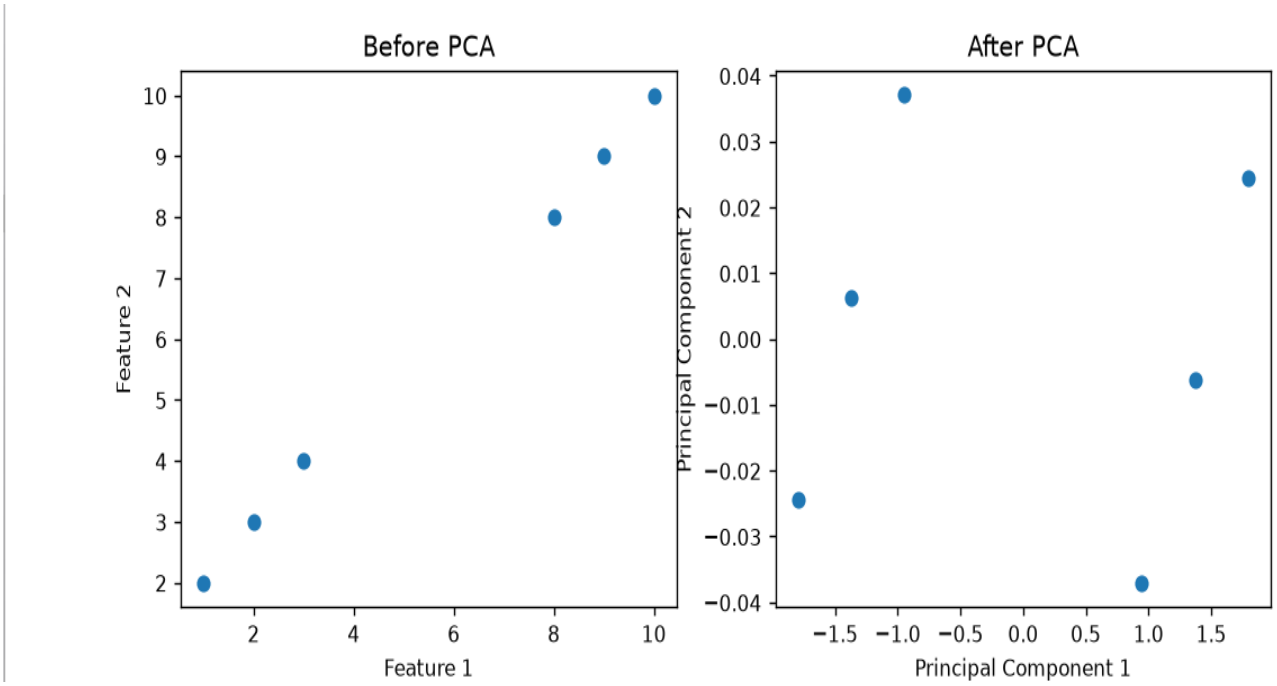
```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1])
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Before PCA")

plt.subplot(1, 2, 2)
plt.scatter(X_pca[:, 0], X_pca[:, 1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("After PCA")
plt.show()
```

Output:



11.Write a python program Create a dataset containing two-dimensional points and display the points using a Matplotlib scatter plot.

Code:

```
import matplotlib.pyplot as plt


x = [1, 2, 3, 8, 9, 10]

y = [2, 3, 4, 8, 9, 10]


plt.scatter(x, y)

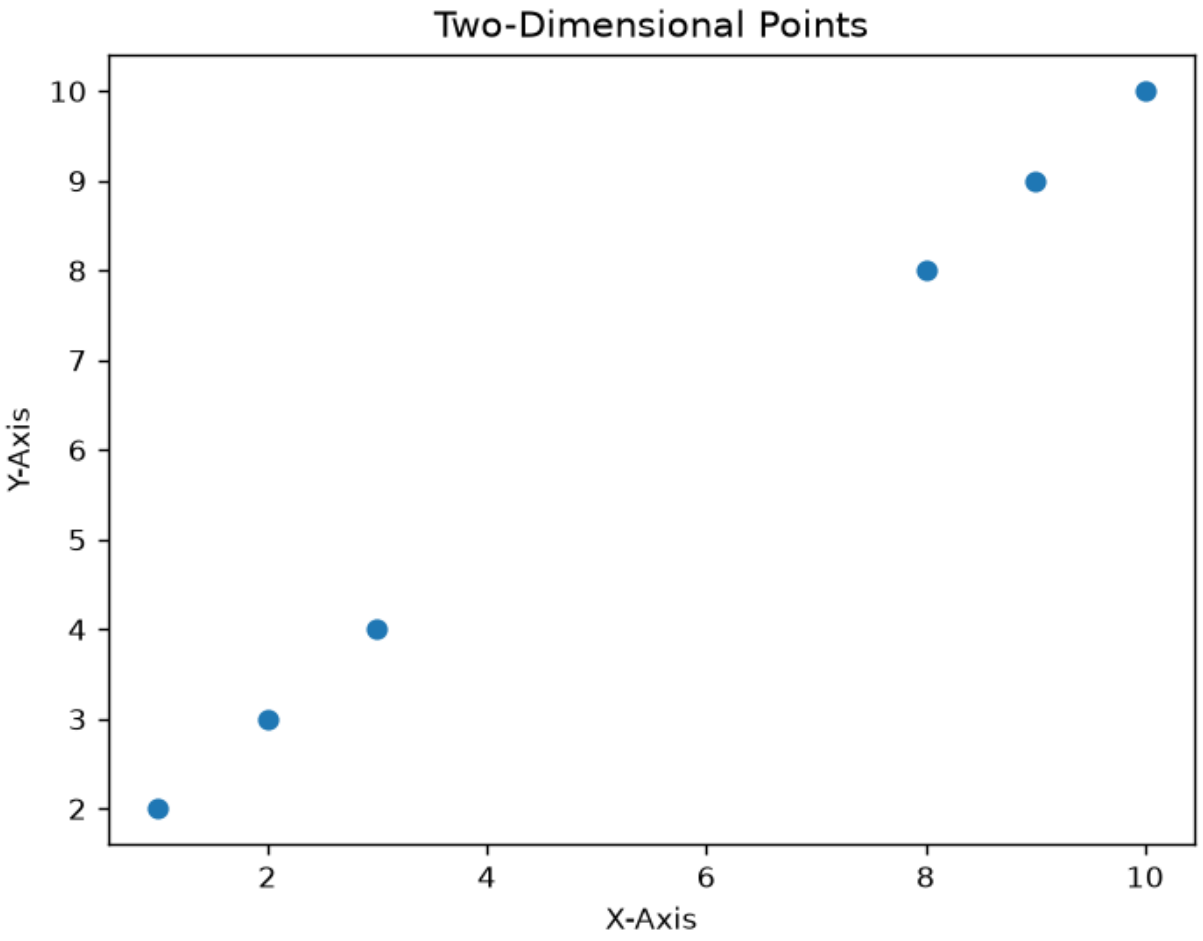
plt.xlabel("X-Axis")

plt.ylabel("Y-Axis")

plt.title("Two-Dimensional Points")

plt.show()
```

Output:



12.Write a python program to Calculate and display the variance of different features in a small dataset using Python.

Code:

```
import pandas as pd


data = {
   "x" : [1, 2, 3, 8, 9, 10],
   "y" : [2, 3, 4, 8, 9, 10]
}


df = pd.DataFrame(data)
variance = df.var()
print("Variance of Different Features:")
print(variance)
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q12.py"
Variance of Different Features:
x    15.5
y    11.6
dtype: float64
```

13.Write a python program create a simple dataset such as house size and price to build a neural network model that predicts the price of a house.

Code:

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense


X = np.array([[1000], [1500], [2000], [2500]])
y = np.array([10, 15, 20, 25])


model = Sequential()
```

```
model.add(Dense(1, input_dim=1))


model.compile(loss='mse', optimizer='adam')


model.fit(X, y, epochs=20)


new_house = np.array([[3000]])
prediction = model.predict(new_house)


print("Predicted house price:", prediction[0][0])
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q13.py"
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1790922810.514308    1360 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors fr
om different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1790922813.688155    1360 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors fr
om different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
C:\Users\patel\AppData\Local\Programs\Python\Python312\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
 a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
I0000 00:00:1790922816.223336    1360 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 o
r the TensorFlow-DirectML plugin.
Epoch 1/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 642ms/step - loss: 4332906.0000
Epoch 2/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step - loss: 4325258.0000
Epoch 3/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 52ms/step - loss: 4317617.5000
Epoch 4/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step - loss: 4309984.0000
Epoch 5/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step - loss: 4302356.5000
Epoch 6/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 71ms/step - loss: 4294737.0000
Epoch 7/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - loss: 4287125.0000
Epoch 8/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step - loss: 4279521.0000
Epoch 9/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 72ms/step - loss: 4271923.0000
Epoch 10/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 75ms/step - loss: 4264335.0000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step
3824.37
```

14.Write a python progarn to Create a simple artificial neuron in Python using input values, weights, and bias. Calculate the output of the neuron.

Code:

```
inputs = [2, 3]

weights = [0.5, 0.8]

bias = 1

output = inputs[0]*weights[0] + inputs[1]*weights[1] + bias

print("Output:", output)
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q14.py"
Output: 4.4
```

15.Write a Python program to apply the Sigmoid, ReLU, and Tanh activation functions on given input values and display their outputs.

Code:

```
import math


x = 2


sigmoid = 1/(1+math.exp(-x))

relu = max(0, x)

tanh = math.tanh(x)


print("Sigmoid:", sigmoid)

print("ReLU:", relu)

print("Tanh:", tanh)
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q15.py"
Sigmoid: 0.8807970779778823
ReLU: 2
Tanh: 0.9640275800758169
```

16.Write a python program to Create a simple feedforward neural network using Keras with an input layer, one hidden layer, and an output layer.

Code:

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(4, input_dim=2, activation='relu'))

model.add(Dense(1, activation='sigmoid'))

model.summary()
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q16.py"
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1790923183.189493   16796 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors fr
om different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1790923186.357243   16796 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors fr
om different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
C:\Users\patel\AppData\Local\Programs\Python\Python312\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
 a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
I0000 00:00:1790923188.714720   16796 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 4)	12
dense_1 (Dense)	(None, 1)	5

```
Total params: 17 (68.00 B)
Trainable params: 17 (68.00 B)
Non-trainable params: 0 (0.00 B)
```

17.Write a python program in program Use a small dataset of student marks and attendance to build a simple neural network that predicts whether a student will pass or fail.

Code:

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense


X = np.array([
    [30, 50],
    [40, 60],
    [50, 70],
    [60, 75],
    [70, 80],
    [80, 90]
])


y = np.array([0, 0, 1, 1, 1, 1])


model = Sequential()


model.add(Dense(4, input_dim=2, activation='relu'))
```

```
model.add(Dense(1, activation='sigmoid'))


model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])


model.fit(X, y, epochs=5, verbose=0)


student = np.array([[65, 75]])


result = model.predict(student)


if result[0][0] >= 0.5:
    print("Student will Pass")
else:
    print("Student will Fail")
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q17.py"
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1790923286.287519    8760 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors fr
om different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1790923289.400015    8760 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors fr
om different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
C:\Users\patel\AppData\Local\Programs\Python\Python312\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
 a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
I0000 00:00:1790923291.743202    8760 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 o
r the TensorFlow-DirectML plugin.
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 71ms/step
Student will Fail
```

18.Write a python program to Create a simple neural network using Keras to classify data into two classes and display the predicted output.

Code:

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense
```

```
X = np.array([
    [1, 2],
    [2, 3],
    [8, 9],
    [9, 10]
])

y = np.array([0, 0, 1, 1])

model = Sequential([
    Dense(1, input_dim=2, activation='sigmoid')
])

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

model.fit(X, y, epochs=10, verbose=0)

prediction = model.predict(np.array([[1.5, 2.5]]))

print("Predicted Output:", prediction)
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q18.py"
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1790923369.922769   22992 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors fr
om different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1790923372.822954   22992 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors fr
om different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
C:\Users\patel\AppData\Local\Programs\Python\Python312\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
 a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
I0000 00:00:1790923375.255902   22992 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 o
r the TensorFlow-DirectML plugin.
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step
Predicted Output: [[0.5688591]]
```

19.Write a payprogram program for Train a simple neural network model using a sample dataset and display the loss value after each training epoch.

Code:

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense


X = np.array([

    [1, 2],

    [2, 3],

    [8, 9],

    [9, 10]

])


y = np.array([0, 0, 1, 1])


model = Sequential([

    Dense(1, input_dim=2, activation='sigmoid')

])


model.compile(

    loss='binary_crossentropy',

    optimizer='adam',
```

```
    metrics=['accuracy']
)

history = model.fit(X, y, epochs=5, verbose=1)

print("\nLoss after each epoch:")

for i, loss in enumerate(history.history['loss']):
    print("Epoch", i + 1, "Loss:", loss)
```

Output:

```
PS D:\BSc Data Science & Analytics\SEM-5\Fundamental of Machine Learning\Pratical Assignment> & C:/Users/patel/AppData/Local/Programs/Python/Python312/python.exe "d:/BSc Data
 Science & Analytics/SEM-5/Fundamental of Machine Learning/Pratical Assignment/Q19.py"
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1790923464.800135    7068 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors fr
om different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1790923467.555445    7068 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors fr
om different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
C:\Users\patel\AppData\Local\Programs\Python\Python312\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
 a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
I0000 00:00:1790923469.889789    7068 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 o
r the TensorFlow-DirectML plugin.
Epoch 1/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 755ms/step - accuracy: 0.0000e+00 - loss: 1.0135
Epoch 2/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step - accuracy: 0.0000e+00 - loss: 1.0094
Epoch 3/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - accuracy: 0.0000e+00 - loss: 1.0053
Epoch 4/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step - accuracy: 0.0000e+00 - loss: 1.0012
Epoch 5/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 73ms/step - accuracy: 0.0000e+00 - loss: 0.9971

Loss after each epoch:
Epoch 1 Loss: 1.0135411024093628
Epoch 2 Loss: 1.0093863010406494
Epoch 3 Loss: 1.0052684545516968
Epoch 4 Loss: 1.0011883974075317
Epoch 5 Loss: 0.9971462488174438
```